

AI BlogNest API – Generate high-quality blog content using AI

Description

The **AI BlogNest API** is a secure and intelligent REST backend for blog creation, content management and AI-assisted writing. It is built with **Node.js** and **Express.js**, uses **MongoDB** as the primary database and **Mongoose ODM** for data modeling and validation.

The platform integrates **Google Gemini AI** for **AI blog generation** and **AI content summarization**, so users can create context-aware content with minimal effort.

Security is built in: **JWT** for stateless authentication, **role-based access control (Admin, Editor, Author, Reader)** for authorization, and **bcrypt.js** for password hashing. Input validation, sanitization, rate limiting and centralized error handling keep the platform reliable and safe from common web vulnerabilities.

Scenario-Based Case Study

Background

Priya is a content creator who manages multiple blogs but struggles to organize posts, manage categories and tags, and track reader engagement.

Problem

- Difficulty managing multiple drafts, categories and tags.
- Limited visibility into post performance and engagement.
- Time-consuming content approval and comment moderation.
- No streamlined handling of media uploads and metadata.
- No permission tiers for editors, authors and administrators.

Solution

AI BlogNest provides a centralized, secure API that separates business logic from frontend delivery. It adds a permission matrix and dedicated services for content lifecycle, categories, media paths, comments and user roles, plus AI tools that draft and summarize posts.

Usage

- **Author:** creates posts, registers media paths, manages drafts, generates blogs with AI.
- **Editor / Admin:** reviews the content queue, changes publishing status, moderates comments, upgrades user roles.
- **Reader:** reads published posts and adds comments.

Outcome

Workflow delays are reduced, schema validation keeps data consistent, security filters ensure only authorized roles perform administrative actions, and AI shortens the time from idea to draft.

1. Software Requirements

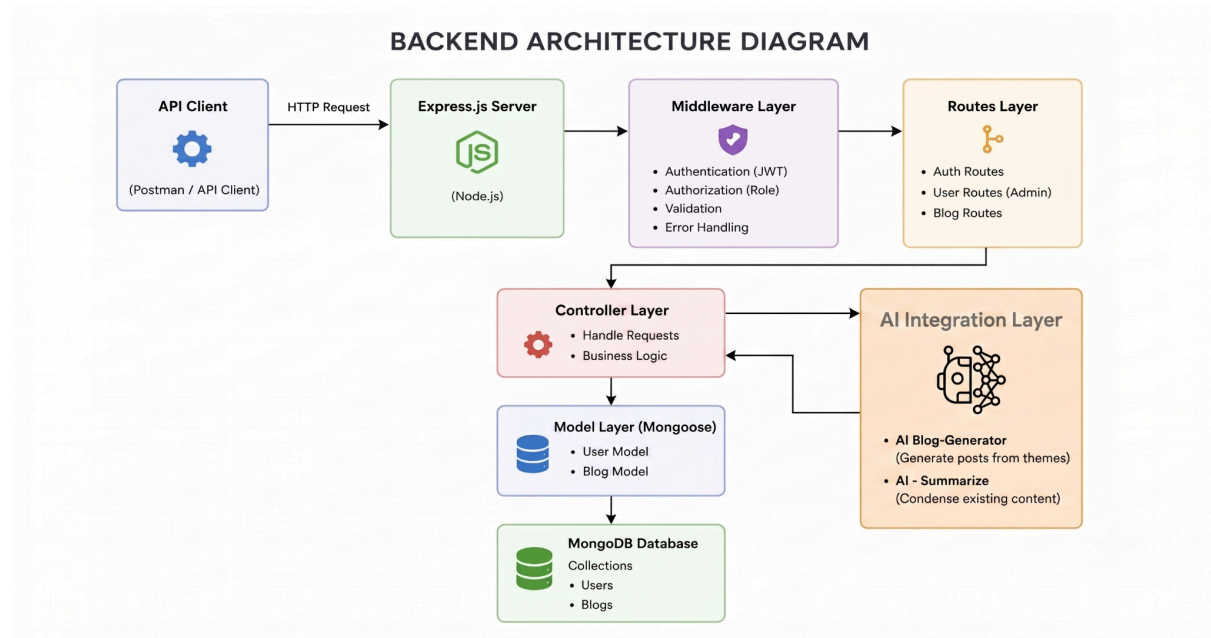
- **Operating System:** Windows 10/11, macOS or Linux.
- **Node.js (v16+):** runtime for the server.
- **npm (v8+):** package manager.
- **Express.js:** web framework for REST routes.
- **MongoDB:** stores users, posts, comments and analytics.
- **Postman / ThunderClient:** API testing.
- **Code Editor:** Visual Studio Code or similar.
- **Google Gemini API key:** for AI features.

2. Hardware Requirements

Component	Requirement
Processor	IntelCore i5 (8th Gen+) / AMD Ryzen 5 or equivalent
RAM	8GB minimum (16 GB recommended)
Storage	1GB free disk space

Epic 1: Project Architecture

Epic:-1 PROJECT ARCHITECTURE

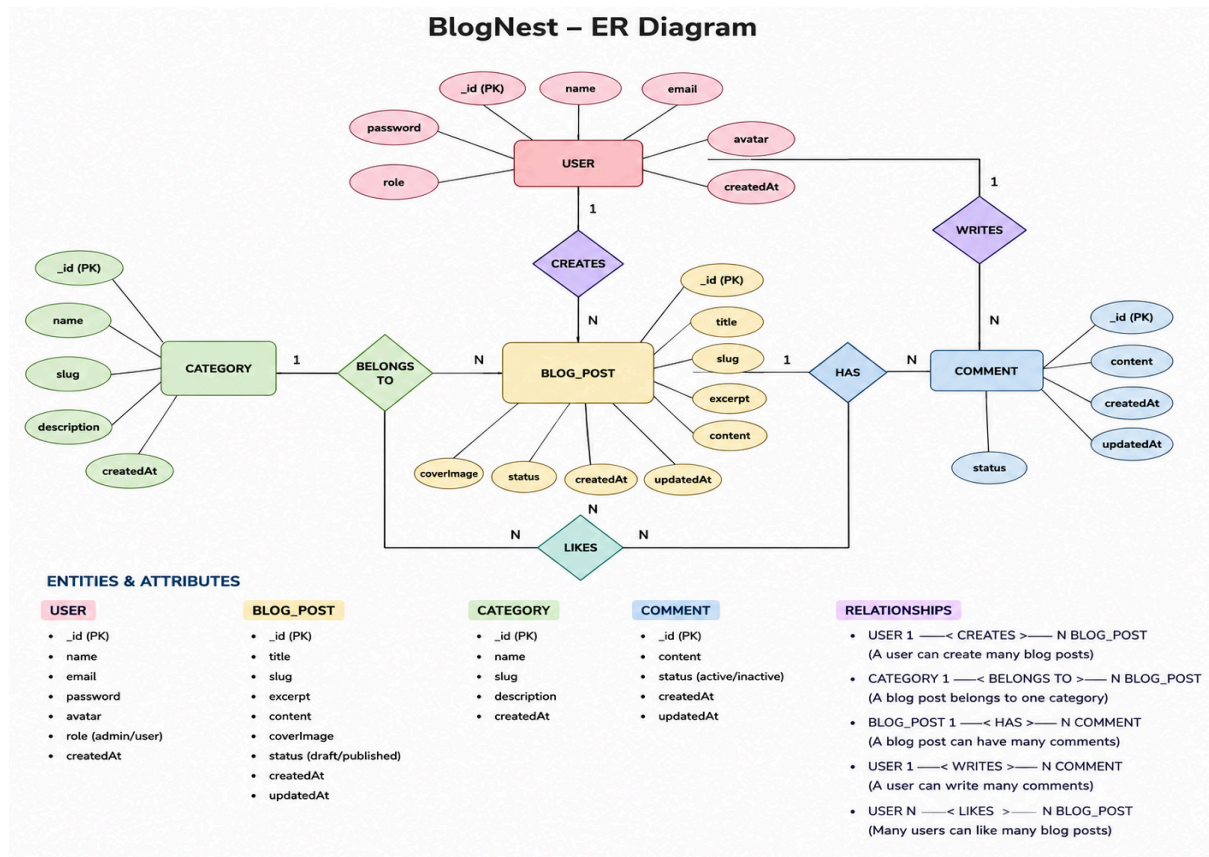


Requests pass through middleware filters before reaching the data services.

- **Interface:** Postman or a frontend calls the route paths and receives JSON.
- **Express Server Gateway:** listens on a port, maps GET/POST/PUT/DELETE, parses JSON and handles CORS.
- **Auth and Role Middleware:** reads the token from headers, verifies it and checks whether the role may access the route.
- **Service / Logic Modules:** blog lifecycle (Draft, Pending Approval, Scheduled, Published), comment queue and reporting.
- **Database Layer:** Mongoose converts application logic into MongoDB queries.
- **AI Flow:** connects to Gemini through `/api/ai/generate-blog` and `/api/ai/summarize`.

• Entity-Relationship Description

The underlying document structure manages several collections bound by logical application references:



1. User

- _id (ObjectId, Primary Key)
- name (String, Required)
- email (String, Required, Unique)
- password (String, Hashed, Required)
- role (String, Default: "reader")

2. Post

- _id (ObjectId, Primary Key)
- userID (Reference to User)
- title (String, Required)
- photo (String, media path)
- message (String, content body)
- likes (Number, Default: 0)
- timestamps (true)

3. Comment

- `_id` (ObjectId, Primary Key)
- `postID` (Reference to Post)
- `userID` (Reference to User)
- `text` (String, Required)
- `status` (approved, pending, spam)

Relationships: User to Post (one-to-many), User to Comment (one-to-many), Post to Comment (one-to-many).

Key Features

- **Role-Based Route Guarding:** token checks block unauthorized roles.
- **CRUD Blog Engine:** full management of posts, status and media paths.
- **Categorization:** tags and categories for filtering and discovery.
- **Comment Moderation:** protected routes to mark comments approved, pending or spam.
- **Sanitization:** prevents XSS and NoSQL injection.
- **AI Blog Generation:** turns a topic into a structured full-length post.
- **AI Summarization:** creates short abstracts of long articles.

Roles and Responsibilities

Role	Responsibilities
Admin	Full control: manage users, change roles, remove malicious comments, view logs
Editor	Review pending drafts, publish posts, clean up categories

Author	Create and edit only their own posts, use AI generation
Reader	Read published posts and comment while logged in

MVC Architecture

- **Model:** Mongoose schemas define fields, defaults, indexes and validation.
- **Controller:** receives requests, validates input, calls models and returns JSON.
- **View (Routing Layer):** a headless API has no templates, so the routes map URLs to controller functions.

Epic 2: Project Setup and Configuration

1. Create a project folder.
2. Inside it create a folder named `server` and open it in VS Code.
3. Run `npm init -y` in the terminal.
4. Install dependencies:

```
npm install express mongoose cors dotenv bcryptjs jsonwebtoken @google/generative-ai
```

5. Create `server.js` and the folders `models`, `controllers`, `routes`, `middleware`.
6. Create a `.env` file:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/blognest
JWT_SECRET=your_secret_key
GEMINI_API_KEY=your_gemini_key
```

Epic 3: Backend Development

server.js

```
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const mongoose = require("mongoose");

const app = express();
app.use(cors());
app.use(express.json());

app.use("/api/auth", require("./routes/authRoutes"));
app.use("/api/posts", require("./routes/postRoutes"));
app.use("/api/ai", require("./routes/aiRoutes"));
```

```
mongoose.connect(process.env.MONGO_URI)
  .then(() => app.listen(process.env.PORT, () =>
    console.log("Server running on port " + process.env.PORT)))
  .catch(err => console.error(err));
```

Auth middleware

```
const jwt = require("jsonwebtoken");

exports.protect = (req, res, next) => {
  const token = (req.headers.authorization || "").split(" ")[1];
  if (!token) return res.status(401).json({ message: "No token" });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch { res.status(401).json({ message: "Invalid token" }); }
};

exports.allow = (...roles) => (req, res, next) =>
  roles.includes(req.user.role) ? next()
  : res.status(403).json({ message: "Access denied" });
```

Epic 4: Database Configuration

Install Mongoose with `npm install mongoose`, connect before the server starts (see `server.js`), then define the schemas.

models/userModel.js

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ["admin", "editor", "author", "reader"], default: "reader" },
}, { timestamps: true });

module.exports = mongoose.model("User", userSchema);
```

models/postModel.js

```
const postSchema = new mongoose.Schema({
  userID: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
  name: { type: String, required: true },
  photo: String,
  message: String,
  likes: { type: Number, default: 0 },
}, { timestamps: true });
```

API Endpoints

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Register a user
POST	/api/auth/login	Public	Login and get JWT
POST	/api/posts	Author, Editor, Admin	Create a blog post
GET	/api/posts	Public	Fetch all posts
GET	/api/posts/:id	Public	Fetch one post
PUT	/api/posts/:id	Owner, Editor, Admin	Update a post
DELETE	/api/posts/:id	Owner, Admin	Delete a post
POST	/api/ai/generate-blog	Author and above	Generate a blog from a topic
POST	/api/ai/summarize	Author and above	Summarize blog content

Epic 5: Project Execution

1. Install Postman or ThunderClient.
2. Start the server with `npm start`.

API Testing

- **User Registration:** POST `/api/auth/register` with name, email, password.
- **User Login:** POST `/api/auth/login`, copy the returned token.
- **Create Blog:** POST `/api/posts` with header `Authorization: Bearer TOKEN`.
- **Fetch Blogs:** GET `/api/posts`.
- **Generate AI Blog:** POST `/api/ai/generate-blog` with body `{ "topic": "Benefits of remote work" }`.
- **Summarize Blog:** POST `/api/ai/summarize` with body `{ "content": "full article text" }`.

Resources

- **Demo Video:**
https://drive.google.com/file/d/1bsT9oZCQv8qW2ZiNCV24g_dtiZwK_efV/view?usp=drivesdk
- **API Testing Video:** add your link here
- **Code Drive:**
<https://drive.google.com/drive/folders/1S69VGxkcmkrsPhfJ6CE9af4Nf9WVnLsj?usp=sharing>